

Software Requirements Specification (SRS)

Project: Chat Application (Socket Programming)

Version: 1.0

Authors: Pranay Shah (PES2UG24CS366); Nikhil Mahabala Shekar (PES2UG24CS318); Prarthana Herur (PES2UG24CS367); Nikhil B Menon (PES2UG24CS317)

Date: 03-09-2026

Status: Draft

Team Members & Contribution

Name	SRN	Contribution
Pranay Shah	PES2UG24CS366	Requirements gathering and stakeholder analysis; Sections 1-2 (Introduction, Overall Description); Section 6 (Quality Attributes & Acceptance Tests); overall document structuring, compilation, formatting, and final review
Nikhil Mahabala Shekar	PES2UG24CS318	Section 4.1-4.3 functional requirements (Authentication, Direct Messaging, Group Chat/Rooms)
Prarthana Herur	PES2UG24CS367	Section 5 (NFRs & Security), Section 7 (UML Use-Case Diagrams) and Section 8 (RTM)
Nikhil B Menon	PES2UG24CS317	Section 3 (External Interfaces) and Section 4.4-4.7 functional requirements (Presence, History, File Sharing, Notifications)

Revision History

Version	Date	Author	Change Summary	Approval
1.0	03-09-2026	Pranay Shah, Nikhil Mahabala Shekar, Prarthana Herur, Nikhil B Menon	Initial SRS drafted for Chat Application project	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Faculty Guide			

Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features (Detailed)
5. Non-Functional Requirements (Detailed)
6. Quality Attributes & Acceptance Tests
7. System Models and Diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a real-time Chat Application built using socket-based communication (WebSockets). It defines the functional and non-functional requirements, external interfaces, and verification criteria for the system, and is intended for use by developers, testers, and evaluators as a reference during design, implementation, and assessment.

1.2 Scope

The Chat Application enables registered users to exchange text messages in real time over persistent WebSocket connections, either one-to-one or within group chat rooms. The system covers client-side chat UI interactions, a server-side application layer (Node.js with the ws/Socket.IO library, or Python Flask-SocketIO), message routing and broadcast, presence tracking, message persistence, and basic administrative moderation. It excludes voice/video calling, end-to-end call signalling, and integration with external enterprise messaging platforms, which are considered out of scope for this version.

1.3 Audience

Developers, QA Engineers, System Integrators, Project Evaluators, and Course Instructors.

1.4 Definitions, Acronyms and Abbreviations

- WS — WebSocket, a full-duplex communication protocol over a single TCP connection
- API — Application Programming Interface
- UI — User Interface
- TLS — Transport Layer Security
- JWT — JSON Web Token, used for stateless session authentication
- DB — Database
- UUID — Universally Unique Identifier
- RTT — Round-Trip Time
- XSS — Cross-Site Scripting
- RTM — Requirements Traceability Matrix

2. Overall Description

2.1 Product Perspective

The Chat Application is a standalone, self-contained client-server system. Clients (web browser or lightweight desktop client) connect to a central chat server over a persistent WebSocket connection. The server is implemented using Node.js with the ws/Socket.IO library, or alternatively Python Flask-SocketIO, and is responsible for connection management, message routing/broadcast, room management, presence tracking, and persistence of chat history to a database.

2.2 Major Product Functions

- User registration and authentication
- Real-time one-to-one (direct) messaging
- Group chat rooms with join/leave/create functionality
- Online/offline/typing presence indicators
- Persistent message history and retrieval
- File and media sharing within a chat
- Push/in-app notifications for new messages
- Administrative moderation (mute, ban, message removal, audit logs)

2.3 User Roles and Characteristics

- Registered User: general chat participant; expects low-latency delivery, an intuitive UI, and reliable message history.
- Room Owner/Moderator: a registered user with elevated privileges over a specific chat room (add/remove members, delete messages in that room).
- Administrator: manages the overall platform — user accounts, global moderation, and audit/logging.

2.4 Operating Environment

Server: Node.js runtime (or Python 3.x with Flask-SocketIO) deployed on a Linux-based cloud VM or container, behind a reverse proxy (e.g. Nginx) terminating TLS. Client: any modern web browser supporting the WebSocket API (Chrome, Firefox, Edge, Safari) or a lightweight cross-platform desktop/mobile client. Database: a relational or NoSQL store (e.g. PostgreSQL / MongoDB / Redis for presence and session caching).

2.5 Constraints

- Must operate over standard HTTP/HTTPS ports with WebSocket upgrade (ws:// or wss://).
- Must support at least 500 concurrent WebSocket connections per server instance for the target deployment.
- Must comply with general data-protection practice for storing user credentials and personal chat data (hashed passwords, encrypted transport).
- Limited to text and common media file types (images, short audio clips, documents up to a configured size limit) for the initial release.

3. External Interface Requirements

3.1 User Interfaces

A responsive web-based chat UI comprising: a login/registration screen, a contact/room list panel, an active conversation panel with a message composer, typing and online-status indicators, and a notification toast/badge. The UI shall follow accessible design practices (sufficient colour contrast, keyboard navigation, screen-reader friendly labels for message controls).

3.2 Hardware Interfaces

- Standard client hardware: keyboard, pointing device/touchscreen, microphone/camera only if optional media capture is used, network interface (Wi-Fi/Ethernet/mobile data).
- Server hardware: standard cloud compute instance with network interface capable of sustaining many concurrent persistent connections.

3.3 Software Interfaces

- WebSocket Gateway API — bidirectional event-based interface (Socket.IO events / ws message frames) for message send, receive, typing, presence, and room events.
- Authentication API — REST endpoints (JSON over HTTPS) for registration, login, and JWT token issuance/refresh.
- Persistence API — internal service layer connecting to the database for storing/retrieving users, rooms, and messages.
- Notification Service — optional integration with a push-notification provider (e.g. Firebase Cloud Messaging) for offline users.

3.4 Communications

- TLS 1.2+ (wss://) enforced for all WebSocket traffic and HTTPS for all REST calls.
- Automatic reconnect with exponential backoff on the client if the WebSocket connection drops; queued outgoing messages are resent on reconnect.
- Heartbeat/ping-pong frames used to detect and clean up stale connections on the server.

4. System Features (Detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow CHAT-F-####.

4.1 User Registration & Authentication

Description: Register new users and authenticate returning users, issuing a secure session token.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-001	The system shall allow a new user to register an account using a unique username/email and a password.	Functional	High	End User	AC-CHAT-F-001: Valid unique registration creates an account and returns success. Test: TC-Auth-01	Password hashing required
CHAT-F-002	The system shall authenticate a returning user via username/email and password and issue a session token (JWT).	Functional	High	End User	AC-CHAT-F-002: Correct credentials return a valid JWT; incorrect credentials are rejected. Test: TC-Auth-02	Depends on CHAT-F-001
CHAT-F-003	The system shall reject login attempts with invalid credentials and lock the account for 5 minutes after 5 consecutive failed attempts.	Functional	High	Security	AC-CHAT-F-003: 6th consecutive failure returns a lockout message. Test: TC-Auth-03	Audit log entry required

4.2 Real-Time Direct Messaging

Description: Establish persistent connections and deliver one-to-one messages in real time, with offline queuing.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-010	The system shall establish a persistent WebSocket connection between an authenticated client and the server upon login.	Functional	High	System	AC-CHAT-F-010: Connection upgrade succeeds and client receives a connection-ack event. Test: TC-Msg-01	Requires CHAT-F-002
CHAT-F-011	The system shall deliver a one-to-one text message from a sender to an online recipient in real time (target < 1s).	Functional	High	End User	AC-CHAT-F-011: Recipient receives the message within the target latency in a test environment. Test: TC-Msg-02	Requires open WS connection
CHAT-F-012	The system shall queue messages for offline recipients and deliver them upon the recipient's next reconnect.	Functional	High	End User	AC-CHAT-F-012: Offline recipient receives all queued messages, in order, after reconnecting. Test: TC-Msg-03	Depends on message persistence

4.3 Group Chat / Chat Rooms

Description: Create, join, and broadcast messages within named group chat rooms.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-020	The system shall allow a user to create a named group chat room and become its owner.	Functional	High	End User	AC-CHAT-F-020: Room is created and creator is listed as owner/member. Test: TC-Room-01	
CHAT-F-021	The system shall allow a user to join an existing public room or a private room via invite.	Functional	High	End User	AC-CHAT-F-021: User appears in room member list after joining. Test: TC-Room-02	
CHAT-F-022	The system shall broadcast a message sent to a room to all currently connected members of that room.	Functional	High	End User	AC-CHAT-F-022: All online members receive the broadcast message. Test: TC-Room-03	Depends on CHAT-F-020/021

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-023	The system shall allow a room owner to remove a member from the room.	Functional	Medium	Room Owner	AC-CHAT-F-023: Removed member loses access to the room and is notified. Test: TC-Room-04	

4.4 Presence & Typing Indicators

Description: Reflect a user's connection and composing activity to relevant contacts in real time.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-030	The system shall display a user's online/offline status to their contacts and room members in real time.	Functional	Medium	End User	AC-CHAT-F-030: Status indicator updates within 3s of a connection state change. Test: TC-Pres-01	
CHAT-F-031	The system shall show a "typing..." indicator to the recipient(s) while a user is composing a message.	Functional	Low	End User	AC-CHAT-F-031: Typing indicator appears/disappears based on composer activity. Test: TC-Pres-02	

4.5 Message History & Persistence

Description: Durably store messages and allow paginated retrieval of past conversation history.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-040	The system shall persist all sent messages (direct and room) with sender, timestamp, and content to the database.	Functional	High	System	AC-CHAT-F-040: Every delivered message has a corresponding persisted record. Test: TC-Hist-01	
CHAT-F-041	The system shall allow a user to retrieve and scroll through the message history of a conversation or room, paginated.	Functional	Medium	End User	AC-CHAT-F-041: Requesting history returns the correct paginated set in chronological order. Test: TC-Hist-02	Depends on CHAT-F-040

4.6 File & Media Sharing

Description: Allow controlled sharing of images and documents within a conversation.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-050	The system shall allow a user to upload and share an image or document (up to a configured size limit) within a conversation.	Functional	Medium	End User	AC-CHAT-F-050: Uploaded file within the limit is shared and visible/downloadable to recipients. Test: TC-Media-01	Storage service required
CHAT-F-051	The system shall reject file uploads that exceed the configured size limit or an disallowed file type.	Functional	Medium	Security	AC-CHAT-F-051: Oversized/disallowed file is rejected with a clear error message. Test: TC-Media-02	

4.7 Notifications

Description: Alert users to new activity both in-app and via push notification when offline.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-060	The system shall show an in-app unread-message badge/count for	Functional	Medium	End User	AC-CHAT-F-060: Badge count increments on new	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	conversations and rooms with new activity.				message and clears on read. Test: TC-Notif-01	
CHAT-F-061	The system shall send a push notification to an offline or backgrounded user when they receive a new direct message.	Functional	Low	End User	AC-CHAT-F-061: Push notification is received on a registered device within a reasonable delay. Test: TC-Notif-02	Requires notification service integration

5. Non-Functional Requirements (Detailed)

NFRs below are measurable and tied to test plans. IDs follow CHAT-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
CHAT-NF-001	End-to-end message delivery latency (sender send to recipient receive, both online) shall be ≤ 1 second for 95% of messages under normal load.	Performance	High	95th percentile ≤ 1 s measured in a load test. Test: TC-Perf-01
CHAT-NF-002	The server shall support at least 500 concurrent active WebSocket connections per instance without degradation beyond the latency target.	Scalability	High	Load test sustains 500 concurrent connections with NF-001 latency intact. Test: TC-Perf-02
CHAT-NF-003	The system shall provide 99.5% monthly uptime, excluding scheduled maintenance windows.	Reliability/Availability	High	Uptime monitoring reports $\geq 99.5\%$ per month. Test: Ops reports
CHAT-NF-004	The chat UI shall be usable on both desktop and mobile-width browsers (responsive layout) without loss of core functionality.	Usability	Medium	Manual/automated UI test across at least 3 viewport sizes passes. Test: TC-UX-01
CHAT-NF-005	The system shall log application errors and key events in a structured format suitable for automated monitoring and alerting.	Maintainability/Observability	Medium	Log review confirms structured, timestamped entries for defined event types. Test: TC-OPS-01

5.1 Security

5.1.1 Security Objectives

- Protect user credentials and session tokens from disclosure or tampering, both in transit and at rest.
- Ensure that only authenticated, authorized users can send messages as themselves, join rooms they are permitted to, or access moderation functions.
- Prevent injection of malicious content (e.g. script injection via chat messages) that could compromise other users' clients.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
CHAT-SR-001	TLS 1.2+ (wss:// / https://) shall be mandatory for all WebSocket and REST network connections.	Security	High	AC-CHAT-SR-001: Plaintext (ws:// / http://) connections are refused by the server. Test: TC-Sec-01
CHAT-SR-002	User passwords shall be stored using a salted, adaptive hashing algorithm (e.g. bcrypt/argon2) and never in plaintext.	Security	High	AC-CHAT-SR-002: Database inspection confirms only salted hashes are stored. Test: TC-Sec-02
CHAT-SR-003	All session/authentication tokens (JWT) shall have a bounded expiry and be validated on every WebSocket message and REST request.	Security	High	AC-CHAT-SR-003: Expired or tampered tokens are rejected with an authentication error. Test: TC-Sec-03

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
CHAT-SR-004	All user-supplied message content shall be sanitised/escaped before rendering to prevent script injection (XSS).	Security	High	AC-CHAT-SR-004: A message containing script tags renders as inert text, not executable script. Test: TC-Sec-04
CHAT-SR-005	Administrative and moderation actions (ban, mute, message deletion) shall be restricted to authorized roles and recorded in an audit log.	Security	Medium	AC-CHAT-SR-005: Non-privileged users are denied access; privileged actions appear in the audit log. Test: TC-Sec-05

6. Quality Attributes & Acceptance Tests

Exit criteria for acceptance: All high-priority functional requirements implemented and verified, no critical NFR or security-requirement failures, and the RTM shows all planned test cases passed.

Acceptance test suites: Registration & Authentication, Direct Messaging, Group Chat/Rooms, Presence, Message History, File Sharing, Notifications, Performance, and Security.

7. System Models and Diagrams

7.1 UML Use-Case Diagrams

Two use-case diagrams are provided below, covering the Registered User and Administrator/Moderator actors and their principal interactions with the Chat Application.

Use-Case Diagram 1: Registered User Interactions

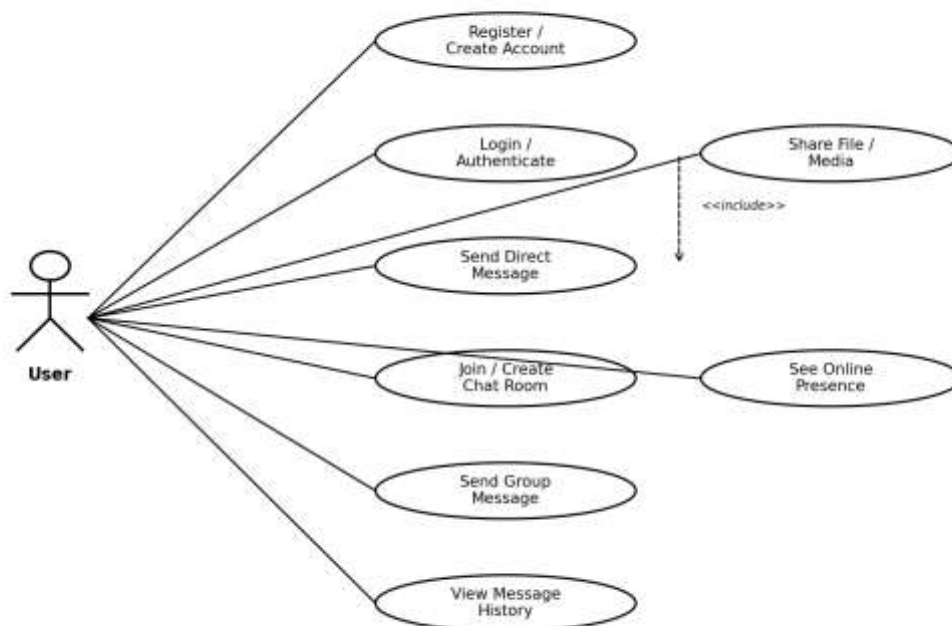


Figure 1: Use-Case Diagram — Registered User Interactions

Use-Case Diagram 2: Administrator / Moderator Interactions

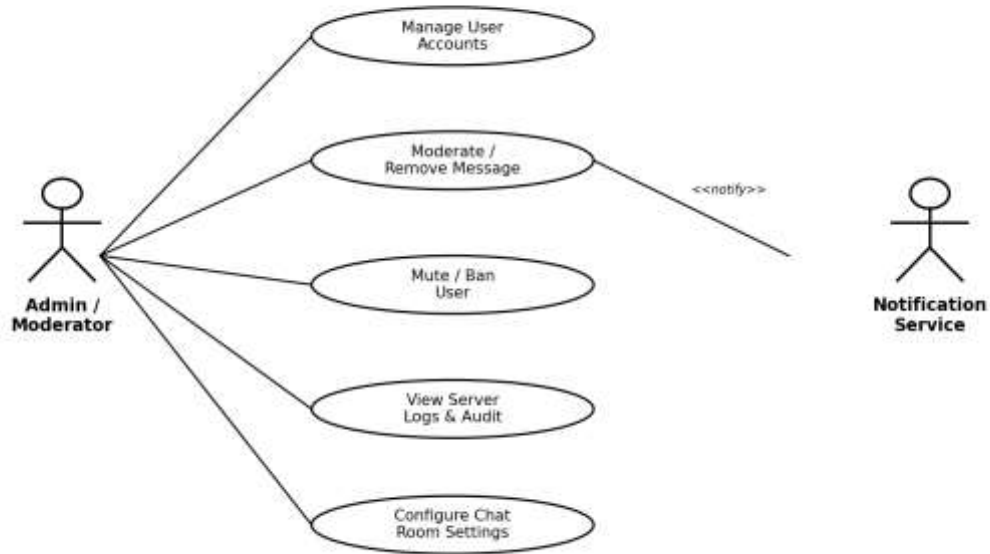


Figure 2: Use-Case Diagram — Administrator / Moderator Interactions

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
CHAT-F-002	Authenticate user	4.1 / DS-Auth-01	AuthModule	TC-Auth-02	N	
CHAT-F-011	Real-time direct message delivery	4.2 / DS-Msg-01	MessagingModule	TC-Msg-02	N	
CHAT-F-022	Broadcast message to room	4.3 / DS-Room-01	RoomModule	TC-Room-03	N	
CHAT-F-041	Retrieve paginated message history	4.5 / DS-Hist-01	HistoryModule / DB	TC-Hist-02	N	
CHAT-NF-001	Message delivery latency target	5 / DS-Perf-01	MessagingModule	TC-Perf-01	N	
CHAT-SR-004	Sanitise message content (XSS)	5.1.2 / DS-Sec-01	MessagingModule / UI	TC-Sec-04	N	